PedalShare

Campus Bicycle Sharing Platform

Complete Project Documentation

Version	1.0
Date	March 2026
Framework	Django 4.2 / Python 3.12
License	MIT (Open Source)
Repository	github.com/U-ROHITH/Pedal-Share
Maintainer	Rohith Uppunthula
Initial Collab	Yamsani Vamshi Krishna

Open Source Project

PedalShare is an open-source project. Anyone — students, developers, or designers — is welcome to read the code, report issues, suggest improvements, and submit pull requests. No approval required to contribute. Fork it, build on it, make it yours.

Built by students, for students — sustainable campus mobility

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Goals . . . 3

2 Architecture Overview **3**
2.1 Technology Stack . . . 3
2.2 High-Level Architecture . . . 3
2.3 Request Flow . . . 4

3 Application Modules **4**
3.1 apps/core — Base & Utilities . . . 4
3.1.1 Base Model . . . 4
3.1.2 Context Processor . . . 5
3.1.3 Static Pages . . . 5
3.1.4 Seed Management Command . . . 5
3.2 apps/users — Authentication & Profiles . . . 5
3.2.1 Models . . . 5
3.2.2 Views & Endpoints . . . 6
3.2.3 Google SSO Flow . . . 6
3.3 apps/cycles — Bicycle Listings . . . 6
3.3.1 Cycle Model . . . 6
3.3.2 Filtering . . . 7
3.3.3 Endpoints . . . 7
3.4 apps/bookings — Rental Bookings . . . 7
3.4.1 Booking Model . . . 7
3.4.2 Endpoints . . . 8
3.5 apps/payments — Wallet & Transactions . . . 8
3.5.1 Transaction Model . . . 8
3.5.2 Payment Flow . . . 8
3.5.3 Endpoints . . . 8
3.6 apps/complaints — Support System . . . 9
3.6.1 Complaint Model . . . 9
3.6.2 Endpoints . . . 9

4 Database Schema **9**

5 Setup & Installation **9**
5.1 Requirements . . . 9
5.2 Local Setup . . . 10
5.3 Environment Variables . . . 10
5.4 Google SSO Configuration . . . 10
5.5 Production Deployment . . . 11

6 URL Reference **11**

7 Contributing **12**
7.1 How to Contribute . . . 12

7.2 Ideas for Contribution 12
7.3 Code Style Guidelines 13
7.4 Reporting Issues 13

8 Security Notes **13**

9 License **13**

10 Acknowledgements **14**

Introduction

PedalShare is a web-based bicycle sharing platform designed for college campuses. Students who own bicycles they do not use every day can list them for rent. Other students can browse available cycles, book one for a few hours, and pay from an in-app wallet. The goal is to reduce the cost of campus commuting, promote sustainable transport, and give cycle owners a way to earn from idle assets.

The platform is built entirely with open-source technology (Django, Bootstrap, SQLite) and is itself open source under the MIT license. Anyone can run it, extend it, and contribute back.

Motivation

- Many students own bicycles that sit unused for most of the day.
- Campus distances are short but walking takes time.
- Existing rental services are expensive or unavailable on campuses.
- A peer-to-peer model directly connects owners with riders at low cost.

Goals

1. Provide a simple, self-hostable platform any campus can deploy.
2. Make sign-in frictionless (Google SSO alongside email/password).
3. Build a fair wallet system so transactions are transparent.
4. Keep the code modular so contributors can improve one area without touching others.
5. Stay open source so the community can extend and improve it freely.

Architecture Overview

Technology Stack

Layer	Technology
Backend	Django 4.2, Python 3.12
Frontend	Bootstrap 4, HTML5, CSS3, Vanilla JavaScript
Authentication	Django built-in auth + Google Identity Services (GIS)
Database	SQLite 3 (development); PostgreSQL-compatible
Static Files	WhiteNoise with CompressedManifestStaticFilesStorage
WSGI Server	Gunicorn
Deployment	Heroku / Render / Railway compatible

High-Level Architecture

The project follows Django's MVT (Model-View-Template) pattern with a modular app structure. Each domain (cycles, bookings, payments, complaints, users) lives in its own

app under the `apps/` directory.

```
PedalShare/
  manage.py
  requirements.txt
  Procfile
  runtime.txt
  project1/              # Django project configuration
    settings.py
    urls.py
    wsgi.py
    asgi.py
  apps/                  # Domain apps (modular)
    core/
    users/
    cycles/
    bookings/
    payments/
    complaints/
  templates/             # Global HTML templates
  static/                # CSS, JS, images
  data/                  # Seed data and historical DB
```

Listing 1: Project layout

Request Flow

1. A request arrives at the Django WSGI application (Gunicorn in production).
2. `project1/urls.py` routes the request to the correct app URLconf.
3. The app's view runs business logic (authenticate, query DB, process form).
4. The view renders a template (from `templates/`) with context data.
5. WhiteNoise serves static assets (CSS, JS, images) directly.

Application Modules

apps/core — Base & Utilities

The `core` app provides shared infrastructure used by all other apps.

Base Model

All domain models inherit from `BaseModel`, which adds:

```
class BaseModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    is_active   = models.BooleanField(default=True)

    class Meta:
```

```
        abstract = True
        ordering = ['-created_at']
```

Listing 2: apps/core/models.py

Context Processor

`apps/core/context_processors.py` injects two variables into every template automatically:

- `GOOGLE_OAUTH_CLIENT_ID` — read from the environment variable.
- `user_profile` — the `UserProfile` object for the current authenticated user (or `None`).

Static Pages

URL	Page
`/about/`	About PedalShare
`/help/`	FAQ and help guide
`/contact/`	Contact form

Seed Management Command

```
python manage.py seed_data
```

Listing 3: Load seed data

Creates sample users, cycles, bookings, and transactions for development and demo purposes.

apps/users — Authentication & Profiles

Models

UserProfile (extends Django's built-in `User`):

Field	Description
`full_name`	Display name
`phone`	Contact number
`avatar`	Profile picture (upload)
`bio`	Short description
`wallet_balance`	In-app currency balance (default: 500)
`total_rides`	Count of completed bookings
`total_earnings`	Sum of rental income earned
`rating`	Average user rating (0.0–5.0)
`is_verified`	Verification flag

Profiles are created automatically via Django signals when a new `User` is saved.

Views & Endpoints

Endpoint	Purpose
`auth/signup/`	Register with email and password
`auth/signin/`	Login (email or username + password)
`auth/signout/`	Logout
`auth/profile/`	View and update profile
`auth/google-callback/`	Google SSO token verification

Google SSO Flow

1. The frontend loads the Google Identity Services (GIS) library.
2. GIS renders a "Sign in with Google" button on the auth pages.
3. On click, GIS returns a signed JWT credential to the browser.
4. The browser POSTs the JWT to `/auth/google-callback/`.
5. Django verifies the JWT using `google-auth` library:

```
idinfo = id_token.verify_oauth2_token(
    token, requests.Request(), CLIENT_ID
)
email = idinfo['email']
```

6. If a user with that email exists, they are logged in. Otherwise a new user is created (with an unusable password).
7. A Django session is established and the user is redirected.

apps/cycles — Bicycle Listings

Cycle Model

Field	Description
`owner`	ForeignKey to User
`title`	Listing title
`description`	Detailed description
`cycle_type`	regular / electric / mountain / hybrid
`color`	Bicycle color
`price_per_hour`	Rental rate (decimal)
`location`	Human-readable location
`latitude / longitude`	Optional GPS coordinates
`available_from`	Start of daily availability window
`available_until`	End of daily availability window
`is_available`	Toggle flag (owner-controlled)
`image`	Uploaded photo
`rating`	Average rating from renters
`total_bookings`	Booking count

registration_number	Optional bike registration ID

Filtering

The `CycleFilterForm` supports filtering the listing page by: cycle type, minimum/maximum price per hour, and location keyword.

Endpoints

Endpoint	Purpose
cycles/	Browse available cycles
cycles/detail/<id>/	Cycle detail page
cycles/add/	Add new listing (login required)
cycles/edit/<id>/	Edit own listing
cycles/delete/<id>/	Delete own listing
cycles/my-cycles/	Owner's cycle dashboard
cycles/toggle-availability/<id>/	Toggle available flag

apps/bookings — Rental Bookings

Booking Model

Field	Description
user	Renter (ForeignKey to User)
cycle	Booked cycle (ForeignKey to Cycle)
booking_date	Date of rental
pickup_time	Planned start time
dropoff_time	Planned end time
status	pending / active / completed / cancelled
total_price	Calculated total cost
payment_status	pending / paid
notes	Optional renter notes

Price Calculation:

```
def calculate_duration(self):
    delta = datetime.combine(date.today(), self.dropoff_time) \
          - datetime.combine(date.today(), self.pickup_time)
    hours = max(delta.total_seconds() / 3600, 1)  # minimum 1 hour
    return round(hours, 2)

def calculate_total_price(self):
    return self.calculate_duration() * self.cycle.price_per_hour
```

Listing 4: Booking price logic

Endpoints

Endpoint	Purpose
bookings/book/<cycle_id>/	Create booking
bookings/my-bookings/	User's booking history
bookings/detail/<id>/	Booking detail
bookings/cancel/<id>/	Cancel pending/active booking
bookings/complete/<id>/	Mark booking as completed

apps/payments — Wallet & Transactions

Transaction Model

Field	Description
user	ForeignKey to User
transaction_type	wallet_topup / booking_payment / refund / earnings
amount	Transaction amount
description	Human-readable label
booking	Optional ForeignKey to Booking
balance_before	Wallet balance before transaction
balance_after	Wallet balance after transaction
reference_id	Unique reference string
status	success / failed / pending

Payment Flow

1. User creates a booking (status: pending, payment: pending).
2. User is redirected to the checkout page.
3. User chooses to pay from wallet (`pay-with-wallet/`).
4. System checks wallet balance $\geq$ total_price.
5. Deducts amount from renter's wallet; credits earnings to owner's wallet.
6. Creates two `Transaction` records (debit + credit).
7. Booking payment_status set to "paid".

Endpoints

Endpoint	Purpose
payments/wallet/	Wallet balance + history
payments/topup/	Top-up wallet
payments/payment-gateway/<id>/	Gateway simulation
payments/process-payment/<id>/	Process payment
payments/checkout/<booking_id>/	Booking checkout

payments/pay-with-wallet/<booking_id>/	Pay via wallet

apps/complaints — Support System

Complaint Model

Field	Description
user	ForeignKey to User
category	cycle_issue / booking_issue / payment_issue / user_behavior / other
subject	Short complaint title
description	Detailed description
cycle	Optional related cycle
booking	Optional related booking
status	open / in_progress / resolved / closed
priority	low / medium / high
resolution_notes	Staff response notes
assigned_to	Staff member handling the ticket

Endpoints

Endpoint	Purpose
complaints/raise/	File a new complaint
complaints/my-complaints/	List user's complaints
complaints/detail/<id>/	View complaint details

Database Schema

All models use SQLite in development and are PostgreSQL-compatible. The database indexes are optimized for the most common query patterns:

- `Cycle`: indexed on `(owner, created_at)` and `(is_available, price_per_hour)`
- `Booking`: indexed on `(user, booking_date)` and `(cycle, status)`
- `Transaction`: indexed on `(user, created_at)` and `(status, created_at)`
- `Complaint`: indexed on `(user, status)` and `(status, created_at)`

Setup & Installation

Requirements

- Python 3.10 or higher
- pip
- A Google Cloud project with OAuth 2.0 credentials (for Google SSO)

Local Setup

```
# 1. Clone the repo
git clone https://github.com/U-ROHITH/Pedal-Share.git
cd Pedal-Share

# 2. Create and activate a virtual environment
python -m venv env
source env/bin/activate          # Windows: env\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Create your .env file
echo "DJANGO_SECRET_KEY=change-me" > .env
echo "DJANGO_DEBUG=True" >> .env
echo "GOOGLE_OAUTH_CLIENT_ID=your-client-id.apps.
   googleusercontent.com" >> .env

# 5. Apply migrations
python manage.py migrate

# 6. (Optional) Load seed data
python manage.py seed_data

# 7. Start the development server
python manage.py runserver 4000
```

Listing 5: Local development setup

Open http://localhost:4000 in your browser.

Environment Variables

Variable	Description
`DJANGO_SECRET_KEY`	Django secret key (required in production)
`DJANGO_DEBUG`	`True` for dev, `False` for production
`DJANGO_ALLOWED_HOSTS`	Comma-separated list of allowed hosts
`GOOGLE_OAUTH_CLIENT_ID`	OAuth 2.0 client ID from Google Cloud Console

Google SSO Configuration

1. Visit Google Cloud Console → APIs & Services → Credentials.
2. Click **Create Credentials** → **OAuth 2.0 Client ID**.
3. Select **Web application**.
4. Under **Authorized JavaScript origins**, add: `http://localhost:4000`
5. Under **Authorized redirect URIs**, add: `http://localhost:4000/auth/google-callback/`

6. Copy the generated Client ID into your `.env` file.

Production Deployment

The project ships with a `Procfile` for Heroku-style platforms:

```
web: gunicorn project1.wsgi --log-file -
```

Listing 6: Procfile

Set `DJANGO_DEBUG=False` and `DJANGO_SECRET_KEY` in production environment variables. WhiteNoise handles static files automatically.

Compatible with: **Heroku**, **Render**, **Railway**, any VPS with Nginx + Gunicorn.

URL Reference

App	Endpoint	Auth
Authentication (apps/users)		
	`auth/signin/`	Public
	`auth/signup/`	Public
	`auth/signout/`	Login
	`auth/profile/`	Login
	`auth/google-callback/`	Public
Cycles (apps/cycles)		
	`cycles/`	Public
	`cycles/detail/<id>/`	Public
	`cycles/add/`	Login
	`cycles/edit/<id>/`	Owner
	`cycles/delete/<id>/`	Owner
	`cycles/my-cycles/`	Login
	`cycles/toggle-availability/<id>/`	Owner
Bookings (apps/bookings)		
	`bookings/book/<cycle_id>/`	Login
	`bookings/my-bookings/`	Login
	`bookings/detail/<id>/`	Login
	`bookings/cancel/<id>/`	Login
	`bookings/complete/<id>/`	Login
Payments (apps/payments)		
	`payments/wallet/`	Login
	`payments/topup/`	Login
	`payments/checkout/<id>/`	Login
	`payments/pay-with-wallet/<id>/`	Login
Complaints (apps/complaints)		
	`complaints/raise/`	Login
	`complaints/my-complaints/`	Login

	complaints/detail/<id>/	Login
Static Pages (apps/core)		
	/	Public
	/about/	Public
	/help/	Public
	/contact/	Public

Contributing

PedalShare is Open Source

PedalShare is licensed under the MIT License. Anyone is welcome to contribute — students, developers, designers, writers. You do not need permission to fork, experiment, or open a pull request. The only requirement is that your contribution is in good faith and follows the project's code style.

How to Contribute

1. **Fork** the repository on GitHub.
2. **Clone** your fork locally.
3. **Create a branch** for your change:

```
git checkout -b feature/your-feature-name
```

4. **Make your changes** and write tests if applicable.
5. **Commit** with a clear message:

```
git commit -m "feat: add map-based cycle search"
```

6. **Push** and open a **Pull Request** on GitHub.

Ideas for Contribution

- Switch database to PostgreSQL and add Dockerfile.
- Integrate a real payment gateway (Razorpay, Stripe).
- Add email notifications for bookings and complaints.
- Build a REST API (Django REST Framework) for a mobile app.
- Add map-based cycle discovery using Leaflet.js or Google Maps.
- Improve the admin dashboard with charts and analytics.
- Write unit and integration tests.
- Add multi-language (i18n) support.
- Improve accessibility (WCAG compliance).

Code Style Guidelines

- Follow PEP 8 for Python code.
- Keep views thin — move business logic to model methods or service functions.
- New features should go in the appropriate `apps/` module.
- Do not commit `.env` files, `db.sqlite3`, or `__pycache__` directories.
- Write descriptive commit messages (use conventional commits if possible).

Reporting Issues

Open a GitHub issue at: github.com/U-ROHITH/Pedal-Share/issues

Include: what you expected, what happened, steps to reproduce, and your environment (OS, Python version, browser).

Security Notes

- The `DJANGO_SECRET_KEY` must be a long random string in production. Never commit it to git.
- The `GOOGLE_OAUTH_CLIENT_ID` is safe to expose in the browser but should be set via environment variable.
- Set `DJANGO_DEBUG=False` in production — debug mode leaks internal information.
- The project has `SECURE_CONTENT_TYPE_NOSNIFF`, `X_FRAME_OPTIONS=DENY`, and HSTS headers pre-configured for production.
- All mutating views require login (`@login_required`) and ownership checks.
- CSRF protection is enabled on all forms. The Google callback is CSRF-exempt because it receives the token from Google's servers, not a form.

License

MIT License

Copyright (c) 2026 PedalShare Contributors
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE

WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Acknowledgements

PedalShare was started as a student project to solve a real campus problem. It has grown through collaboration and open-source contributions. Thank you to every person who filed an issue, submitted a pull request, or shared the project.

Project Team

Role	Name	GitHub
Repo Maintainer	Rohith Uppunthula	github.com/U-ROHITH
Initial Collaborator	Yamsani Vamshi Krishna	github.com/YVK49

Rohith Uppunthula created and maintains the repository, driving the core architecture and overall direction of the project. Yamsani Vamshi Krishna contributed the initial Google SSO integration, project restructuring into modular apps, and the full UI redesign — laying the foundation for the platform as it stands today.

Built by students, for students.
Come build with us — github.com/U-ROHITH/Pedal-Share